

Submission Details

Link to Implementation

Public Google Colab notebook:

https://colab.research.google.com/github/satyamdas03/a2-fraud-detection/blob/main/notebook/fraud_detection.ipynb

GitHub repository:

<https://github.com/satyamdas03/a2-fraud-detection>

A2 Project Proposal: Credit Card Fraud Detection

1. Task Definition

1.1 Problem Statement

Build a machine learning system that flags fraudulent credit card transactions in a stream of payment events. The system must output a fraud probability for each transaction so that investigators can prioritize cases.

1.2 Why This Task Matters

Fraud losses on credit and debit cards are estimated in the tens of billions of dollars annually. Manual review of every transaction is impossible at scale. A learning-based system can rank transactions by risk, letting analysts focus effort where it has the highest financial impact.

1.3 Why Machine Learning Is Appropriate

- The input space is high-dimensional and noisy.
- Fraud patterns evolve; rule-based systems become stale.

- The cost structure is asymmetric: missing a fraud is far more expensive than investigating a false alarm.
- Historical labeled data exists, so supervised learning is feasible.

1.4 Input / Output Specifications

Training Phase

Interface	Specification
Input	A table of historical credit card transactions. Each row is a transaction described by numeric features: <code>Time</code> (seconds since first transaction), <code>V1 – V28</code> (PCA-transformed principal components), <code>Amount</code> (transaction value in USD), and a binary label <code>Class</code> (0 = legitimate, 1 = fraud).
Output	A trained classifier that maps a feature vector to a fraud probability.
Valid input	A pandas DataFrame or NumPy array with 30 feature columns and one label column.
Expected output format	A scikit-learn compatible estimator object with <code>fit</code> , <code>predict</code> , and <code>predict_proba</code> methods.

Deployment Phase

Interface	Specification
Input	A single transaction or batch of transactions represented by the same 30 feature columns used during training.
Output	A fraud probability score in the range [0, 1] for each input transaction.
Decision rule	A threshold is chosen on a validation set. Scores above the threshold are flagged for review.

1.5 Dataset

The ULB Machine Learning Group Credit Card Fraud Detection dataset: 284,807 transactions from European cardholders in September 2013, with 492 confirmed frauds (0.172%). Features are PCA-

transformed for privacy. The notebook downloads the CSV directly from a public URL using `urllib.request.urlretrieve`, so it is self-contained and reproducible on Google Colab.

1.6 Research Question

Can a cost-sensitive gradient-boosted classifier, calibrated with a threshold chosen by validation, outperform naive accuracy-based baselines on this heavily imbalanced fraud detection task?

2. Learning Framework

2.1 Hypothesis Space

The model family is gradient-boosted decision trees. Each tree partitions the feature space into axis-aligned regions and assigns a fraud probability. The ensemble combines many weak trees into a strong classifier.

2.2 Loss Function

Training optimizes binary cross-entropy:

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$$

where y is the true label and p is the predicted fraud probability. This loss is well-suited to probabilistic binary classification and naturally outputs calibrated scores.

2.3 Task Objective vs. Loss Function

The business objective is to minimize total fraud loss. That is not exactly cross-entropy. A missed fraud may cost hundreds of dollars; a false alarm costs an investigator's time. This mismatch is addressed by:

- Applying class weights during training.
- Optimizing the classification threshold on a validation set using a custom cost-weighted metric.
- Reporting precision, recall, F1, AUROC, and average precision alongside accuracy.

2.4 Learning Algorithm

Gradient boosting builds trees sequentially. Each new tree predicts the negative gradient of the loss with respect to the current ensemble's predictions. The algorithm is implemented through the LightGBM library, with hyperparameters tuned by random search on the validation set.

3. Evaluation Strategy

3.1 Data Splits

- Train: first 60% of transactions by time.
- Validation: next 20% of transactions by time.
- Test: final 20% of transactions by time.

A temporal split prevents data leakage from future transactions into training, which matters for real-world deployment.

3.2 Metrics

Metric	Role
AUROC	Measures ranking quality across all thresholds. Robust to class imbalance.
Average Precision (AP)	Focuses on the rare positive class; more informative than AUROC on imbalanced data.
Precision@k	Of the top-k flagged transactions, what fraction are actually fraud.
Recall@threshold	What fraction of frauds are caught at a chosen operating point.
Custom cost	$(\text{cost_FN} * \text{FN} + \text{cost_FP} * \text{FP}) / \text{total_transactions}$, used to choose threshold.

3.3 Validation Procedure

- Hyperparameter search uses 5-fold cross-validation on the training set.
- The final model is retrained on the full training set with the best hyperparameters.
- The threshold and cost weights are selected using the validation set.
- Test set is used only once for final unbiased evaluation.

Machine Learning Approach

1. Model Choice and Justification

We use gradient boosted decision trees implemented in LightGBM. The reasoning:

- **Tabular data:** Fraud detection runs on structured features. Tree ensembles outperform neural networks on small-to-medium tabular problems (Grinsztajn et al., 2022).
- **Non-linearity:** Individual trees capture feature interactions without hand-engineered products.
- **Scalability:** LightGBM trains quickly on 284k rows and handles class imbalance via `class_weight`.
- **Interpretability:** Feature importance and SHAP values can explain individual predictions, which matters in regulated financial settings.

The model family is an ensemble of decision trees:

$$F(x) = \sum_{m=1}^M \eta * h_m(x)$$

where each `h_m` is a decision tree and `eta` is the learning rate. The final fraud probability is passed through a sigmoid.

2. Hypothesis Space and Parameterization

The hypothesis space is all functions produced by the LightGBM training algorithm given the hyperparameter ranges we search. Key parameters:

Parameter	Role
<code>num_leaves</code>	Complexity of each tree. More leaves = finer partitions, higher variance.
<code>max_depth</code>	Maximum tree depth. Constrains interaction order.
<code>n_estimators</code>	Number of trees in the ensemble. More trees can overfit if learning rate is too high.
<code>learning_rate</code>	Shrinkage applied to each new tree. Lower values need more trees.
<code>min_child_samples</code>	Minimum samples in a leaf. Regularizes small leaves.
<code>subsample</code> / <code>colsample_bytree</code>	Stochastic sampling of rows/features per tree. Reduces overfitting.

Parameter	Role
<code>class_weight="balanced"</code>	Adjusts loss gradients to compensate for rarity of fraud.

These map directly to the code in `model.py` and to the `RandomizedSearchCV` block in the notebook.

3. Loss Function and Training Procedure

Training minimizes binary cross-entropy:

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$$

This is the natural loss for probabilistic binary classification. In LightGBM the gradients are:

$$\begin{aligned} g_i &= p_i - y_i \\ h_i &= p_i (1 - p_i) \end{aligned}$$

Each new tree is fit to the negative gradient, and the leaf outputs are adjusted by the Hessian. This second-order optimization is why gradient boosting converges faster than first-order methods.

`class_weight="balanced"` reweights samples inversely by class frequency, so the gradient of a fraud example contributes more than a legitimate example. This is one way to handle imbalance; we combine it with threshold tuning.

4. Evaluation Metrics and Their Relevance

Metric	Why it is used
AUROC	Threshold-independent ranking quality. Useful when the operating point is unknown.
Average Precision (AP)	Area under the precision-recall curve. More sensitive to the rare class than AUROC.
Precision / Recall / F1	Standard point-wise metrics at the chosen threshold.

Metric	Why it is used
Total cost	Custom metric that weights false negatives 10x more than false positives, matching business impact.
Precision@100	Of the 100 highest-risk transactions, how many are fraud. Directly relevant to investigator capacity.

5. Validation Scheme

- **Temporal split:** 60% train, 20% validation, 20% test, ordered by `Time`. This prevents look-ahead leakage and mimics production deployment.
- **Cross-validation:** 3-fold CV on the training set is used during `RandomizedSearchCV`.
- **Single test use:** The test set is evaluated only after threshold selection, giving an unbiased estimate.

6. Data Preprocessing

- `Time` is dropped because it is only useful for splitting, not as a feature.
- `Amount` is standardized with `StandardScaler` fit on the training set.
- `V1 – V28` are already PCA-transformed in the original dataset, so no further scaling is applied.
- No SMOTE or synthetic oversampling is used; we rely on class weights and threshold tuning to avoid generating unrealistic synthetic fraud examples.

Results and Discussion

1. Experimental Setup

- Dataset: ULB Credit Card Fraud Detection (284,807 transactions, 492 frauds, 0.172% fraud rate).
- Splits: temporal 60/20/20 by `Time`.
- Model: LightGBM with class balancing, hyperparameters tuned by randomized search (15 candidates, 3-fold CV).
- Threshold: chosen on validation set to minimize total cost with `cost_FN = 100`, `cost_FP = 10`.

2. Hyperparameter Search Results

Best parameters from RandomizedSearchCV :

```
{'colsample_bytree': 1.0,
  'learning_rate': 0.175,
  'max_depth': 4,
  'min_child_samples': 199,
  'n_estimators': 583,
  'num_leaves': 33,
  'subsample': 1.0}
```

Best cross-validated average precision: **0.8464**.

The search preferred a shallow tree depth (max_depth=4) and large min_child_samples=199 , suggesting the model benefits from strong regularization rather than complex partitions. This is consistent with the heavy class imbalance: deep trees could overfit to the few fraud examples.

3. Test Set Performance

Metric	Value
Accuracy	0.9993
Precision	0.7108
Recall	0.7867
F1	0.7468
AUROC	0.9660
Average Precision	0.8064
Total cost (cost_FN=100, cost_FP=10)	1840
Cost per transaction	0.0323
Threshold	0.004

Metric	Value
Precision@100	0.60

4. Interpretation

- **Accuracy is misleading:** 99.93% accuracy looks excellent, but a trivial classifier that always predicts "legitimate" would score 99.83%. Precision, recall, and AP are more honest.
- **Ranking quality is strong:** AUROC of 0.966 and AP of 0.806 show the model separates frauds from legitimate transactions well.
- **Operating point is aggressive:** The threshold 0.004 is low because missing a fraud costs 10x more than a false alarm. This pushes recall above precision, catching ~79% of frauds while flagging ~29% of flagged cases as false positives.
- **Top-100 precision is 60%:** If investigators can only review 100 transactions, 60 are likely fraud. This is a practical deployment metric.

5. Loss Function vs. Task Objective

The training loss is binary cross-entropy; the business objective is minimizing fraud cost. These are not identical. Cross-entropy produces calibrated probabilities but does not encode the asymmetric cost of false negatives. We bridge the gap in two ways:

1. **Class weights** during training increase the influence of fraud examples.
2. **Threshold optimization** after training selects the operating point that minimizes cost on validation data.

A limitation remains: cross-entropy treats all frauds as equally costly and all false alarms as equally cheap. In reality, a \$10,000 fraudulent transaction is worse than a \$1 fraud, and repeated false alarms on VIP customers carry reputational cost. A more refined objective would use instance-specific costs.

6. Limitations

- **Dataset age:** The data is from 2013. Modern fraud patterns differ.
- **Anonymized features:** V1 – V28 are PCA components, so interpretation beyond feature importance is limited.
- **Static model:** Fraudsters adapt. The model would need periodic retraining.

- **No concept drift detection:** We do not monitor distribution shifts between training and live data.

7. Future Work

- Add real-time monitoring of feature distributions and model performance.
- Experiment with instance-weighted losses that account for transaction amount.
- Investigate ensemble methods that combine LightGBM with anomaly detection for emerging fraud patterns.
- Calibrate probabilities on the validation set and report confidence intervals.
- Include investigator feedback loop to continuously retrain on confirmed frauds.

Implementation Log

1. Initial Design Decisions

I started by reading the A2 specification and the subject outline. The key constraints were:

- Option 2 requires a real, non-trivial dataset.
- The notebook must be self-contained and runnable on a cloud platform.
- The assessment is 50% code/report and 20% presentation/peer review; attendance at A3 is mandatory.
- AI tools are allowed but must be documented and critically reviewed.

I chose credit card fraud detection because it has a clear business objective, a well-known public dataset, and an obvious mismatch between training loss (cross-entropy) and business objective (asymmetric fraud cost). That mismatch gives something concrete to analyze for Criterion C.

2. Use of AI Tools

I used Claude Code as the main AI assistant for this project. The interactions fell into four categories:

2.1 Specification interpretation and project planning

I pasted the A2 PDF and subject outline into Claude and asked for a recommendation between Option 1 and Option 2. Claude recommended Option 2 with a real-world dataset and a custom

evaluation metric. I accepted this recommendation because it aligned with the rubric: Option 2 allows library use while still requiring deep justification of data/model alignment, and a custom cost metric directly targets the "Evaluation and Refinement" criterion.

2.2 Code generation

I asked Claude to generate:

- A `data_loader.py` module for temporal splitting and scaling.
- A `model.py` wrapper around LightGBM with randomized hyperparameter search.
- An `evaluation.py` module with cost-sensitive threshold optimization and plotting.
- A self-contained Colab notebook.

I then ran the full pipeline locally and checked that the shapes, class counts, and metrics were reasonable. For example, I verified that the temporal split produced training/validation/test fraud counts close to 360/57/75, matching the overall 0.172% fraud rate.

2.3 Report drafting

I asked Claude to draft the report sections using a direct, human-like tone. I then edited the drafts to remove generic phrasing, inserted the actual numeric results from the pipeline, and added specific design justifications (e.g., why a temporal split matters for fraud detection).

2.4 Critical review and verification

Every AI-generated claim was checked against the code or external sources:

- The LightGBM binary cross-entropy gradient formulas were cross-checked with the LightGBM documentation.
- The dataset source and size (284,807 rows, 492 frauds) were verified by loading the CSV directly.
- The claim that tree ensembles outperform neural networks on tabular data was supported by Grinsztajn et al. (2022), "Why do tree-based models still outperform deep learning on typical tabular data?"

3. Challenges and Solutions

3.1 Downloading the dataset inside the notebook

The original ULB dataset is hosted on Kaggle, which requires authentication. A public notebook cannot ask the user for Kaggle credentials. I found a direct public URL to the same CSV (`https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv`) and used `urllib.request.urlretrieve` in the notebook. I verified locally that the file size and row count matched the known dataset.

3.2 Hyperparameter search time

A full grid search over the LightGBM space would take too long in a 15-minute presentation demo. I used `RandomizedSearchCV` with only 15 candidates and 3-fold CV. This keeps runtime under a few minutes while still exploring the main hyperparameters.

3.3 Class imbalance and metric choice

Accuracy on this dataset is always above 99%, so it is not useful. I added AUROC, average precision, F1, and a custom cost metric. The custom cost weights false negatives 10x more than false positives, which forces the model to prioritize catching fraud over avoiding false alarms.

3.4 Threshold optimization

The default 0.5 threshold is inappropriate for imbalanced data. I wrote a function that scans thresholds on the validation set and picks the one minimizing total cost. This is a post-training step, which separates probability calibration from decision making.

4. Knowledge Gaps and Verification

Topic	Confidence	How verified
LightGBM gradient formulas	High	Official docs + reproduced in report
Temporal split vs. random split	High	Domain knowledge; no code bug because fraud rate stays consistent across splits
Cost-weighted threshold selection	High	Implemented from scratch; checked monotonicity of cost function
PCA feature interpretation	Low	Features are anonymized; I explicitly state this limitation

Topic	Confidence	How verified
Exact real-world cost ratios	Medium	Used illustrative ratio (10:1); noted this should be set by business

5. Reflection

The most valuable part of the project was designing the evaluation around the business objective rather than just accuracy. If I had stopped at accuracy, the model would look perfect while missing most frauds. The biggest risk for the presentation is explaining the threshold choice clearly, because peers may expect a default 0.5 cutoff. I will spend time in the slides explaining why 0.004 is the right operating point given the cost structure.

6. Tools and Libraries Used

- Python 3.10
- NumPy, pandas, scikit-learn, LightGBM, SciPy, Matplotlib
- Claude Code for code generation, planning, and writing assistance
- Google Colab target platform for the public notebook

Presentation Materials

A3 Presentation: Credit Card Fraud Detection

Slide 1: Title

Credit Card Fraud Detection with Cost-Sensitive Gradient Boosting

UTS 32513 — Advanced Data Analytics Algorithms Assessment 2 / 3

Goal: flag fraudulent card transactions from historical payment data.

Slide 2: Problem Definition

- Input: transaction record with `Time`, `V1 – V28` (PCA features), `Amount`.
- Output: fraud probability in `[0, 1]`.
- Decision: flag transactions above a learned threshold for investigator review.
- Why ML? Fraud patterns evolve; rules become stale; labeled historical data exists.

Dataset: 284,807 transactions, only 492 frauds (0.172%).

Slide 3: Data Pipeline

- Temporal split: 60% train / 20% validation / 20% test, ordered by `Time`.
- `Amount` standardized using training statistics.
- `Time` used only for splitting, not as a model feature.

This prevents leakage from future transactions and matches production deployment.

Slide 4: Model and Training

- Model: LightGBM gradient boosted decision trees.
- Loss: binary cross-entropy.
- Class weighting: `balanced` to compensate for rare frauds.
- Hyperparameters tuned by randomized search (15 candidates, 3-fold CV) using average precision.

Best settings: depth 4, 583 trees, learning rate 0.175, large leaf regularization.

Slide 5: Evaluation and Threshold

Training loss $\neq$ business objective.

- Missing fraud is far more expensive than a false alarm.
- We optimize the classification threshold on validation data to minimize total cost:

$$\text{total_cost} = 100 * \text{FN} + 10 * \text{FP}$$

Chosen threshold: 0.004, not 0.5.

Slide 6: Test Results

Metric	Value
AUROC	0.966
Average Precision	0.806
Precision	0.711
Recall	0.787
F1	0.747
Accuracy	0.999
Precision@100	0.60

Accuracy is misleading; ranking and cost metrics matter more.

Slide 7: Limitations and Future Work

- Dataset is from 2013; modern fraud differs.
 - Features are anonymized PCA components.
 - Static model; needs retraining for concept drift.
 - Future: instance-specific costs by transaction amount, real-time monitoring, feedback loop.
-

Anticipated Q&A

Q: Why not use SMOTE to balance the data? A: SMOTE creates synthetic fraud examples that may not follow real feature distributions. Class weighting and threshold tuning avoid inventing fake data.

Q: Why is the threshold so low (0.004)? A: Because false negatives cost 10x more than false positives. A low threshold catches more frauds at the cost of more false alarms.

Q: Why LightGBM instead of a neural network? A: For tabular data of this size, tree ensembles generally train faster and perform better than small neural nets, and they are easier to interpret.